Learning to Optimize Distributed Optimization: ADMM-based DC-OPF Case Study

Meiyi Li[1], Soheil Kolouri[2], Javad Mohammadi[1]
[1]*Department of Civil, Architectural, and Environmental Engineering, The University of Texas at Austin*
[2]*Department of Computer Science, Vanderbilt University*
meiyil@utexas.edu, soheil.kolouri@vanderbilt.edu, javadm@utexas.edu

2023 IEEE Power & Energy Society General Meeting (PESGM) | 978-1-6654-6441-3/23/$31.00 ©2023 IEEE | DOI: 10.1109/PESGM52003.2023.10253438

***Abstract*—The decision-making paradigms of future energy systems are increasingly becoming decentralized and multi-entity/agent. The Alternating Direction Method of Multipliers (ADMM) has been widely used to address the computational needs of decentralized decision-making problems, e.g., optimal power flow (OPF). In this paper, we propose a novel data-driven method to accelerate the convergence of ADMM for decentralized DC-OPF, where our optimizer will learn the iterative behavior of agents to produce a high-quality feasible solution. The proposed method utilizes the gauge maps to enforce feasibility with respect to agents' local constraints while iteratively penalizing violations of the shared constraints. We used the IEEE 57-bus system to showcase the performance of the proposed method. Our experimental results demonstrate significant run-time reduction for using ADMM to solve the DC-OPF problem.**

***Index Terms*—Alternating Direction Method of Multipliers (ADMM), DC optimal power flow, neural network, machine learning, learning to optimize**

NOMENCLATURE

$\mathcal{N}_{\texttt{Bus}}$	set of all buses
Ω^b	set of buses connected with bus b
$\Omega^{b,j}$	set of buses connected to bus b and owned by agent j
$P^b_{\texttt{G}}$	power generation at bus b
$P^b_{\texttt{D}}$	load at bus b
$\overline{P}^b_{\texttt{G}}$	upper bound of power generation at bus b
$\underline{P}^b_{\texttt{G}}$	lower bound of power generation at bus b
θ^b	voltage angle at bus b
$x^{b,\bar{b}}$	line reactance between bus b and bus $\bar{b}$
$\overline{P}^{b,\bar{b}}$	upper flow bound for the line between buses b and $\bar{b}$
$\underline{P}^{b,\bar{b}}$	lower flow bound for the line between buses b and $\bar{b}$
$\mathcal{N}_{\texttt{A}}$	set of all agents
$\mathcal{N}^i_{\texttt{Bus}}$	set of all buses owed by agent i
$\mathcal{N}^i_{\texttt{Bd}}$	set of boundary buses owed by agent i
$\theta^{b,\bar{b}}_{\texttt{Copy}}$	copy of $\theta^{\bar{b}}$ owned by agent i
$\mathcal{S}^i_{\texttt{Local}}$	local constraints set of agent i
$\mathbf{u}^i$	stacked variables of $[P^b_{\texttt{G}}, \theta^b, \theta^{b,\bar{b}}_{\texttt{Copy}}]$ for all $b \in \mathcal{N}^i_{\texttt{Bus}}$
$\mathbf{x}^i$	stacked input parameters $[P^b_{\texttt{D}}]$ for all $b \in \mathcal{N}^i_{\texttt{Bus}}$
ρ	positive penalty coefficient
$\boldsymbol{\lambda}$	stacked variables of all the Lagrangian multipliers $\lambda^{b,\bar{b}}$
$\boldsymbol{\lambda}^i$	stacked Lagrangian multipliers of agent i
$\mathbf{u}^{i[k]}_{\texttt{Other}}$	coupled variables associated with neighboring agents and needed by agent i to solve local subproblems
$\mathbf{u}^i_{\texttt{Dep}}$	agent i's dependent variables
$\mathbf{u}^i_{\texttt{Ind}}$	agent i's independent variables
$\mathcal{S}^i_{\texttt{Local,Ref}}$	reformulated local constraints set
$\hat{\mathbf{u}}^i_{\texttt{Ind}}$	intermediate prediction of $\mathbf{u}^i_{\texttt{Ind}}$ in unit box $\mathcal{B}$
$\mathbf{u}^i_{\texttt{Ind},0}$	interior point of $\mathcal{S}^i_{\texttt{Local,Ref}}$
$\mathcal{S}^i_{\texttt{Local,Ref0}}$	the shifted set of $\mathcal{S}^i_{\texttt{Local,Ref}}$ by the interior points $\mathbf{u}^i_{\texttt{Ind},0}$
$N_{\texttt{K}}$	total iteration number of ADMM process
k	ADMM's iteration index
$N_{\texttt{R}}$	number of recurrent steps for training

I. INTRODUCTION

The future electric power grid will be distributed and needs to cope with the widespread adoption of renewable energy sources, facilitate demand response participation and enable energy sharing. This decentralization of energy generation and energy sharing requires revisiting the decision-making frameworks. This need, combined with the appeal of efficiency and privacy, has motivated researchers to find decentralized solutions for critical functionalities of the power grid, such as Optimal Power Flow (OPF). Moreover, machine learning (ML) based solutions have shown tremendous potential to improve the efficiency of optimization procedures. In this paper, we study the role of ML-based solutions in distributed optimization problems.

Many of the matured applications of ML for solving the OPF are centralized. In these setups, the entire system shares a neural approximator to derive nodal-level optimal decisions. Examples of these methods include the predict-and-reconstruct approach [3], the primal-dual method [4], and the active constraints method [5]. These ML-based methods seek high-quality, efficient solutions to use for near-real-time operations.

More recently, some efforts have proposed decentralized learning methods where agents could make decisions based on local information [6] without communication among agents. A learning method is proposed in [7] to achieve decentralized energy management for time-varying grid topology. The cloud-based cooperative information exchange framework is adopted to learn OPF in [8] to prevent agents from sharing raw information. Compared with centralized learning methods, these decentralized learning approaches maintain the security of private decision variables. Most existing ML-based approaches that directly regress the optimal arguments have shown to be effective in finding near-optimal solutions but have limited capabilities in finding feasible solutions. Hence, a feasibility restoration step is required (relying on commercial solvers) to ensure feasibility, i.e., to enforce power flow constraints hold. We denote these approaches as "directly-learning" methods.

In contrast to these directly-learning methods, reference [9] and [10] proposed learning-based methods where the iterative

978-1-6654-6441-3/23/$31.00 ©2023 IEEE
Authorized licensed use limited to: FH Technikum Wien. Downloaded on May 11,2026 at 14:32:48 UTC from IEEE Xplore. Restrictions apply.

nature of decentralized algorithms is kept and communication among agents is allowed. The proposed accelerated ADMM methods for OPF enable information sharing after each agent's prediction. This helps with improving the feasibility of coupled constraints. However, these studies adopt penalty-based methods for agents' local constraints, and still, they do not provide any feasibility guarantees.

In this paper, we propose a learning-based method to accelerate the convergence of ADMM for solving the DC-OPF problem. In our multi-agent setup, each agent may control one or a collection of nodes. The proposed method allows each agent to predict the convergence value of local variables by communicating with neighbors. All agents collaborate to provide a near-optimal solution and satisfy node-level and system-level constraints. Unlike restoration-based methods [3], our approach does not require post-processing steps to improve feasibility because local constraints are enforced by design through a gauge mapping method [2], and coupled constraints are penalized by ADMM iterations.

The paper is organized as follows. Section II reviews the ADMM-based decentralized formulation of the DC-OPF problem. In Section III, we present the proposed learning method to accelerate ADMM as well as its design structure and training approach. Section IV showcases the results, and Section V concludes our work.

II. Distributed DC-OPF Formulation via ADMM

The DC-OPF problem aims to minimize the energy procurement cost while satisfying local (generation and line flow limits) and system constraints (i.e., supply and demand balance).

$$\min \sum_{b\in\mathcal{N}_{\text{Bus}}} f^b(P^b_{\text{G}}) \tag{1a}$$

$$\texttt{generation limit:}\ \underline{P}^b_{\text{G}} \le P^b_{\text{G}} \le \overline{P}^b_{\text{G}} \tag{1b}$$

$$\texttt{power balance:}\ P^b_{\text{G}} - P^b_{\text{D}} = \sum_{\bar{b}\in\Omega^b} \frac{\theta^b - \theta^{\bar{b}}}{x^{b,\bar{b}}} \tag{1c}$$

$$\texttt{line limit:}\ \underline{P}^{b,\bar{b}} \le \frac{\theta^b - \theta^{\bar{b}}}{x^{b,\bar{b}}} \le \overline{P}^{b,\bar{b}} \tag{1d}$$

In this formulation, (1b)-(1d) need to hold for all $b \in \mathcal{N}_{\text{Bus}}$. For the distributed DC-OPF problem, the network is decomposed into multiple zones, each controlled by an agent $i \in \mathcal{N}_{\text{A}}$. Then, each agent's sub-problem aims to minimize its local operational costs. Note that constraints (1c) and (1d) may include variables owned by different agents, e.g., $b \in \mathcal{N}^i_{\text{Bus}}$ and $\bar{b} \in \mathcal{N}^j_{\text{Bus}}$. In this case, bus b is defined as a *boundary bus* of agent i (i.e. $b \in \mathcal{N}^i_{\text{Bd}}$) and agent i and agent j are considered *neighbors*. Given that (1c) and (1d) include variables from multiple agents, the above problem (1) cannot be decomposed into disjoint problems for which the independent solutions yield the global minimum. The ADMM deals with the difficulty of finding a decentralized solution for (1) by creating local copies of the "coupled" variables and seeking agreement between copies during the course of iterations [1]. Therefore, we introduce $\theta^{b,\bar{b}}_{\text{Copy}}$, which is owned by agent i, and represents a copy of $\theta^{\bar{b}}$. Then, the distributed DC-OPF sub-problem of agent i can be expressed as (2):

$$\min F^i = \sum_{b\in\mathcal{N}^i_{\text{Bus}}} f^b(P^b_{\text{G}}) \tag{2a}$$

$$\texttt{s.t.}\ \underline{P}^b_{\text{G}} \le P^b_{\text{G}} \le \overline{P}^b_{\text{G}}, \forall b \in \mathcal{N}^i_{\text{Bus}} \tag{2b}$$

$$P^b_{\text{G}} - P^b_{\text{D}} = \sum_{\bar{b}\in\Omega^{b,i}} \frac{\theta^b - \theta^{\bar{b}}}{x^{b,\bar{b}}} + \sum_{\bar{b}\in\Omega^{b,j}, j\neq i} \frac{\theta^b - \theta^{b,\bar{b}}_{\text{Copy}}}{x^{b,\bar{b}}}, \forall b \in \mathcal{N}^i_{\text{Bus}} \tag{2c}$$

$$\underline{P}^{b,\bar{b}} \le \frac{\theta^b - \theta^{\bar{b}}}{x^{b,\bar{b}}} \le \overline{P}^{b,\bar{b}}, \forall b \in \mathcal{N}^i_{\text{Bus}}, \bar{b} \in \Omega^{b,i} \tag{2d}$$

$$\underline{P}^{b,\bar{b}} \le \frac{\theta^b - \theta^{b,\bar{b}}_{\text{Copy}}}{x^{b,\bar{b}}} \le \overline{P}^{b,\bar{b}}, \forall b \in \mathcal{N}^i_{\text{Bus}}, \bar{b} \in \Omega^{b,j}, j \neq i \tag{2e}$$

$$\theta^{b,\bar{b}}_{\text{Copy}} = \theta^{\bar{b}}, \forall b \in \mathcal{N}^i_{\text{Bd}}, \bar{b} \in \Omega^{b,j}, j \neq i \tag{2f}$$

Here, (2f) is the only constraint that includes variables from different agents because $\theta^{b,\bar{b}}_{\text{Copy}}$ is owned by agent i but $\theta^{\bar{b}}$ is owned by agent j. Let us define $\mathcal{S}^i_{\texttt{Local}}$ as the set of local constraints associated with agent i, i.e., (2b)-(2e). Here, $\mathbf{u}^i$ denotes the stacked variables of $[P^b_{\text{G}}, \theta^b, \theta^{b,\bar{b}}_{\text{Copy}}]$ for all $b \in \mathcal{N}^i_{\text{Bus}}$, and $\mathbf{x}^i$ represents the stacked input parameters $[P^b_{\text{D}}]$ for all $b \in \mathcal{N}^i_{\text{Bus}}$. Thus the compact form of the DC-OPF problem (2) simplifies to (3);

$$\min_{\mathbf{u}^i\in\mathcal{S}^i_{\texttt{Local}}(\mathbf{x}^i)} \quad \sum_{i\in\mathcal{N}_{\text{A}}} F^i(\mathbf{u}^i) \tag{3a}$$

$$\texttt{s.t.} \quad \theta^{b,\bar{b}}_{\text{Copy}} = \theta^{\bar{b}}, b \in \mathcal{N}^i_{\text{Bd}}, \bar{b} \in \Omega^{b,j}, j \neq i \tag{3b}$$

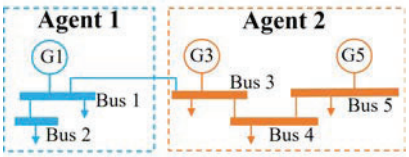

Fig. 1. A two-agent DC-OPF system example. Here Bus 1 is agent 1's boundary bus, and Bus 3 is agent 2's boundary bus, i.e., $\mathcal{N}^1_{\text{Bd}} = \{1\}, \mathcal{N}^2_{\text{Bd}} = \{3\}$. $\mathbf{u}^1 = [P^1_{\text{G}}, \theta^2, \theta^1, \theta^{1,3}_{\text{Copy}}]^T$, $\mathbf{u}^2 = [P^3_{\text{G}}, \theta^4, P^5_{\text{G}}, \theta^3, \theta^5, \theta^{3,1}_{\text{Copy}}]^T$.

The standard form of ADMM solves problem (3) by dealing with the augmented Lagrangian function L:

$$L([\mathbf{u}^i, i \in \mathcal{N}_{\text{A}}], \boldsymbol{\lambda}) = \sum_{i\in\mathcal{N}_{\text{A}}} (F^i(\mathbf{u}^i) + \sum_{\text{all } \lambda^{b,\bar{b}}\in\boldsymbol{\lambda}} \left(\lambda^{b,\bar{b}}(\theta^{b,\bar{b}}_{\text{Copy}} - \theta^{\bar{b}}) + \rho(\theta^{b,\bar{b}}_{\text{Copy}} - \theta^{\bar{b}})^2\right)) \tag{4a}$$

$$\mathbf{u}^i \in \mathcal{S}^i_{\texttt{Local}}(\mathbf{x}^i), \forall i \in \mathcal{N}_{\text{A}} \tag{4b}$$

Here $\rho > 0$ is a positive constant, and $\boldsymbol{\lambda}$ denotes the vector of stacked variables of all Lagrangian multipliers $\lambda^{b,\bar{b}}$, and $\lambda^{b,\bar{b}}$ which correspond with coupled constraint $\theta^{b,\bar{b}}_{\text{Copy}} = \theta^{\bar{b}}$.

The search for a solution to (4) is performed through an iterative process (indexed by $[k], k = 1, 2, ...N_{\text{K}}$). All N_{A} agents will execute this process simultaneously and independently. At the system level, these updates manifest themselves as follows,

$$\mathbf{u}^{i^{[k+1]}} = \underset{\mathbf{u}^i\in\mathcal{S}^i_{\texttt{Local}}(\mathbf{x}^i)}{\arg\min}\ L(\mathbf{u}^i, [\mathbf{u}^{j^{[k]}}, j \neq i, j \in \mathcal{N}_{\text{A}}], \boldsymbol{\lambda}^{[k]}) \tag{5}$$

$$\lambda^{b,\bar{b}^{[k+1]}} = \lambda^{b,\bar{b}^{[k]}} + \rho(\theta^{b,\bar{b}^{[k+1]}}_{\text{Copy}} - \theta^{\bar{b}^{[k+1]}}) \tag{6}$$

978-1-6654-6441-3/23/$31.00 ©2023 IEEE
Authorized licensed use limited to: FH Technikum Wien. Downloaded on May 11,2026 at 14:32:48 UTC from IEEE Xplore. Restrictions apply.

Note that agent i doesn't need to access all the elements in $\mathbf{u}^{j^{[k]}}, \boldsymbol{\lambda}^{[k]}$ to solve (5). It merely needs the variables, which are coupled with it's own variables, i.e. $\theta^{\bar{b}^{[k]}}, \theta^{\bar{b},b^{[k]}}_{\texttt{Copy}}, \lambda^{\bar{b},b^{[k]}}, \lambda^{b,\bar{b}^{[k]}}, b \in \mathcal{N}^i_{\texttt{Bd}}, \bar{b} \in \Omega^{b,j}, j \neq i$. Hence, let $\boldsymbol{\lambda}^{i^{[k]}}$ denote all the $[\lambda^{\bar{b},b^{[k]}}, \lambda^{b,\bar{b}^{[k]}}]$ above, and $\mathbf{u}^{i^{[k]}}_{\texttt{Other}}$ denotes all the coupled variables which are owned by other agents but are needed by agent i to solve (5). Then, (5) reduces to (7).

$$\mathbf{u}^{i^{[k+1]}} = \underset{\mathbf{u}^i \in \mathcal{S}^i_{\texttt{Local}}(\mathbf{x}^i)}{\arg\min} \; L(\mathbf{u}^i, \mathbf{u}^{i^{[k]}}_{\texttt{Other}}, \boldsymbol{\lambda}^{i^{[k]}}) \tag{7}$$

Furthermore, to compute the update function (6), agent i needs to share the updated $\theta^{b,\bar{b}^{[k+1]}}_{\texttt{Copy}}$ and $\theta^{b^{[k+1]}}$ with the corresponding neighbor agent j who owns bus $\bar{b}$. For example, agent 2 in the system shown in Fig. 1 needs $\mathbf{u}^{2^{[k]}}_{\texttt{Other}} = \{\theta^{1^{[k]}}, \theta^{1,3^{[k]}}_{\texttt{Copy}}\}$ from agent 1 to update $\boldsymbol{\lambda}^{2^{[k+1]}} = \{\lambda^{1,3^{[k+1]}}, \lambda^{3,1^{[k+1]}}\}$. See Fig. 2 for a detailed implementation of ADMM.

The standard form of ADMM guarantees the feasibility of local constraints by (7) and penalizes violations of coupled constraints $\theta^{b,\bar{b}}_{\texttt{Copy}} = \theta^{\bar{b}}$ by iterative updating Lagrangian multipliers as (6). In what follows, we will propose a learning-based method to accelerate ADMM for decentralized DC-OPF. The iterative nature of ADMM is kept in the proposed method to improve the feasibility of coupled constraints, while the gauge mapping method [2] is adopted to enforce hard local constraints.

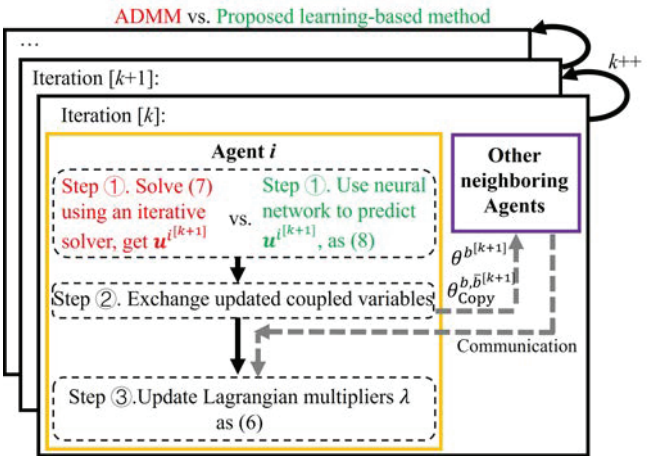

Fig. 2. ADMM versus the proposed learning-based method. Rather than using an iterative solver, the proposed method utilizes neural networks which can directly map the input to the convergence value in a single feed-forward.

III. Learning-accelerated ADMM

A. Method overview

This section provides a high-level overview of our proposed method to incorporate an ML method to accelerate the ADMM algorithm. Instead of solving the $N_{\texttt{A}}$ local optimization problems (7) by an iterative solver, we will train $N_{\texttt{A}}$ neural networks $\xi_{\mathbf{w}^i}, i \in \mathcal{N}_{\texttt{A}}$ to directly map the input to the convergence value of local variables in a single feed-forward. We use ★ to denote the resulting prediction. Here, we replace (7) with (8), and ideally $\mathbf{u}^{i^{[k+1]\bigstar}}$ will be close to optimal solution of (3).

$$\mathbf{u}^{i^{[k+1]\bigstar}} = \xi_{\mathbf{w}^i}\left(\mathbf{x}^i, \mathbf{u}^{i^{[k]}}_{\texttt{Other}}, \boldsymbol{\lambda}^{i^{[k]}}\right) \tag{8}$$

Pseudo code of the proposed learning method for ADMM is given in Algorithm 1. The proposed method includes three steps for each iteration. First, each agent uses its neural approximator to predict the local optimal solution. Second, agents share the updated coupled variables with neighboring agents. Finally, Lagrangian multipliers will be updated according to (6).

Algorithm 1 Proposed learning method for ADMM

Input: DCOPF ADMM problem parameters, e.g., $N_{\texttt{A}}$ neural network $\xi_{\mathbf{w}^i}$; Initial value of $\mathbf{u}^i, i \in \mathcal{N}_{\texttt{A}}$, $\boldsymbol{\lambda}$.
Output: Distributed solution $\mathbf{u}^{i^{[k+1]}}$ to DC-OPF.
while Convergence criteria unmet **do**
  for $i \in \mathcal{N}_{\texttt{A}}$ **do**
    Generate a prediction $\mathbf{u}^{i^{[k+1]\bigstar}} = \xi_{\mathbf{w}^i}(\mathbf{x}^i, \mathbf{u}^{i^{[k]}}_{\texttt{Other}}, \boldsymbol{\lambda}^{i^{[k]}})$
    Receive $\mathbf{u}^{i^{[k+1]}}_{\texttt{Other}}$ from all neighboring agents
    Update Lagrangian multipliers $\boldsymbol{\lambda}^{i^{[k+1]}}$
    $k++$
  end for
end while

B. Neural network structure designing

According to the standard ADMM form (7) and (6), iterative updates of Lagrangian multipliers penalize violations of coupled constraints while the results produced by (7) satisfies local constraints. Therefore, we will keep the ADMM iterations to improve the feasibility of coupled constraints. Further, we will design each neural network's structure to guarantee its output satisfies the local constraints, i.e., $\xi_{\mathbf{w}^i}\left(\mathbf{x}^i, \mathbf{u}^{i^{[k]}}_{\texttt{Other}}, \boldsymbol{\lambda}^{i^{[k]}}\right) \in \mathcal{S}^i_{\texttt{Local}}(\mathbf{x}^i)$. Therefore, we adopt the $\mathcal{LOOP}-\mathcal{LC}$ model proposed in [2] for each neural network $\xi_{\mathbf{w}^i}$. The $\mathcal{LOOP}-\mathcal{LC}$ model learns to solve optimization problems with hard linear constraints. By applying variable elimination and gauge mapping, the $\mathcal{LOOP}-\mathcal{LC}$ model could produce a feasible and near-optimal solution. The application of the $\mathcal{LOOP}-\mathcal{LC}$ model to the proposed method (as shown in Fig.3) is explained below:

1) Variable elimination: The network's output $\mathbf{u}^i$ includes $[P^b_{\texttt{G}}], [\theta^b], [\theta^{b,\bar{b}}_{\texttt{Copy}}]$. According to equality constraints (2c), we divide the variables into two parts where **dependent variables** $\mathbf{u}^i_{\texttt{Dep}}$ are determined by **independent variables** $\mathbf{u}^i_{\texttt{Ind}}$. For example, consider agent 1 in Fig.1. $P^1_{\texttt{G}}$ and θ^2 can be regarded as dependent variables relying on θ^1 and $\theta^{1,3}_{\texttt{Copy}}$ as $P^1_{\texttt{G}} = P^1_{\texttt{D}} + P^2_{\texttt{D}} + \frac{\theta^1 - \theta^{1,3}_{\texttt{Copy}}}{x^{1,3}}$ and $\theta^2 = \theta^1 - x^{1,2}P^2_{\texttt{D}}$. Therefore, once θ^1 and $\theta^{1,3}_{\texttt{Copy}}$ are determined, $P^1_{\texttt{G}}$ and θ^2 can be obtained.

Let $\mathbb{F}^i$ denote the relationship between $\mathbf{u}^i_{\texttt{Dep}}$ and $\mathbf{u}^i_{\texttt{Ind}}$, i.e., $\mathbf{u}^i_{\texttt{Dep}} = \mathbb{F}^i(\mathbf{u}^i_{\texttt{Ind}})$. Take $\mathbb{F}^i$ into the definition of $\mathcal{S}^i_{\texttt{Local}}$ and replace $\mathbf{u}^i_{\texttt{Dep}}$, we could reformulate the problem (7) to a reduced-size problem whose variable is $\mathbf{u}^i_{\texttt{Ind}}$. Denote the reformulated local constraints set as $\mathcal{S}^i_{\texttt{Local,Ref}}$.

978-1-6654-6441-3/23/$31.00 ©2023 IEEE
Authorized licensed use limited to: FH Technikum Wien. Downloaded on May 11,2026 at 14:32:48 UTC from IEEE Xplore. Restrictions apply.

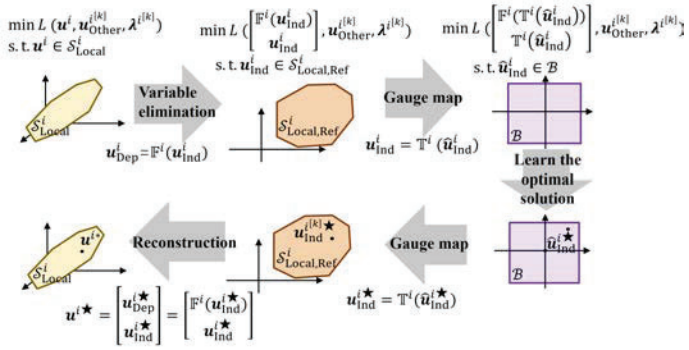

Fig. 3. We adopt the $\mathcal{LOOP}-\mathcal{LC}$ model proposed in [2] for each neural network $\xi_{\mathbf{w}^i}, i \in \mathcal{N}_{\mathrm{A}}$. First, we divide the variables into two parts where dependent variables $\mathbf{u}^i_{\mathtt{Dep}}$ are determined by independent variables $\mathbf{u}^i_{\mathtt{Ind}}$ according to local equality constraints. We reformulate the problem (7) to a reduced-size problem only relying on $\mathbf{u}^i_{\mathtt{Ind}}$. Then we relax the reformulated problem to a problem whose variables are $\hat{\mathbf{u}}^i_{\mathtt{Ind}}$ with only upper and lower bounds. Then, use neural network to predict the optimal solution $\hat{\mathbf{u}}^{i\bigstar}_{\mathtt{Ind}}$. Finally, we will obtain a $\mathbf{u}^{i\bigstar} \in \mathcal{S}^i_{\mathtt{Local}}$ through gauge mapping and reconstruction.

Therefore, we produce the prediction of the reformulated problem and make sure $\mathbf{u}^i_{\mathtt{Ind}} \in \mathcal{S}^i_{\mathtt{Local,Ref}}$. Then, we will apply $\mathbb{F}^i$ to produce a full-size $\mathbf{u}^i = [\mathbf{u}^i_{\mathtt{Dep}}, \mathbf{u}^i_{\mathtt{Ind}}] \in \mathcal{S}^i_{\mathtt{Local}}(\mathbf{x}^i)$.

2) Gauge map: Our target becomes to predict $\mathbf{u}^i_{\mathtt{Ind}}$ and make sure it satisfies $\mathcal{S}^i_{\mathtt{Local,Ref}}$. Instead of solving it directly, we train the neural network to find a virtual prediction $\hat{\mathbf{u}}^i_{\mathtt{Ind}}$ in the ℓ_∞-norm unit ball $\mathcal{B}$, a constraint set with only upper and lower bound. Layers of the neural network will ensure that the resulting $\hat{\mathbf{u}}^i_{\mathtt{Ind}}$ stays within $\mathcal{B}$. Later we will provide a one-to-one gauge mapping to transfer $\hat{\mathbf{u}}^i_{\mathtt{Ind}}$ from $\mathcal{B}$ to $\mathcal{S}^i_{\mathtt{Local,Ref}}$. This mapping will be denoted as $\mathbb{T}^i$. According to [2], $\mathbb{T}^i$ is a known function with a closed form as obtained from (9).

$$\mathbf{u}^i_{\mathtt{Ind}} = \mathbb{T}^i(\hat{\mathbf{u}}^i_{\mathtt{Ind}}) = \frac{\psi_{\mathcal{B}}(\hat{\mathbf{u}}^i_{\mathtt{Ind}})}{\psi_{\mathcal{S}^i_{\mathtt{Local,Ref0}}}(\hat{\mathbf{u}}^i_{\mathtt{Ind}})}\hat{\mathbf{u}}^i_{\mathtt{Ind}} + \mathbf{u}^i_{\mathtt{Ind},0} \tag{9}$$

Here $\psi_{\mathcal{B}}$ is the Minkowski function on set $\mathcal{B}$. $\mathbf{u}^i_{\mathtt{Ind},0}$ is an interior point of $\mathcal{S}^i_{\mathtt{Local,Ref}}$. $\mathcal{S}^i_{\mathtt{Local,Ref0}} = \left\{\bar{\mathbf{u}}^i_{\mathtt{Ind}} \middle| \left(\mathbf{u}^i_{\mathtt{Ind},0} + \bar{\mathbf{u}}^i_{\mathtt{Ind}}\right) \in \mathcal{S}^i_{\mathtt{Local,Ref}}\right\}$ is a shifted set.

To sum up, the $\mathcal{LOOP}-\mathcal{LC}$ model allows the neural network to find a virtual prediction $\hat{\mathbf{u}}^{i\bigstar}_{\mathtt{Ind}}$. Then, apply $\mathbb{T}^i$ as (9) and $\mathbb{F}^i$, we can obtain a prediction $\mathbf{u}^{i\bigstar} \in \mathcal{S}^i_{\mathtt{Local}}(\mathbf{x}^i)$.

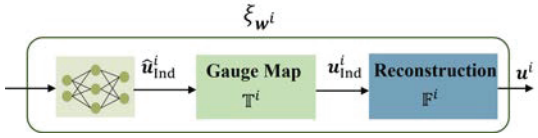

Fig. 4. We adopt the $\mathcal{LOOP}-\mathcal{LC}$ model proposed in [2] for each neural network to make sure $\xi_{\mathbf{w}^i} \in \mathcal{S}^i_{\mathtt{Local}}(\mathbf{x}^i)$.

C. Neural network training

Note that predicting converged ADMM values is a time-series prediction problem where the outputs from the last iteration will serve as the inputs for upcoming iterations. That is, $\mathbf{u}^{i[k+1]\bigstar}$ depends on $\mathbf{u}^{i[k]\bigstar}_{\mathtt{Other}}$ (obtained from other agents' output $\mathbf{u}^{j[k]\bigstar}$ of the last iteration). The training process should look ahead and consider future steps. Hence, we will jointly train all neural networks in a recurrent fashion that allows previous outputs of other agents to be used as inputs. The training flow chart is illustrated in Fig. 5.

Assume there are N training data points, and we index them and their corresponding output by an superscript (n). First, we will run ADMM to obtain all the $\mathbf{u}^{i[k](n)}$ and $\boldsymbol{\lambda}^{[k](n)}$ for training. Also, we have the optimal solution $\mathbf{u}^{i*(n)}$ of (3). Then, we consider N_{R} recurrent steps. The loss function is the sum of distance d between the prediction and the optimal solution over all the agents, all recurrent steps, all iterations $(k = 1, ..N_{\mathrm{K}})$, and all data points, as in (10).

$$f_{\mathrm{L}} = \sum_{n=1}^{N}\sum_{k=1}^{N_{\mathrm{K}}}\sum_{r=1}^{N_{\mathrm{R}}}\sum_{i\in\mathcal{N}_{\mathrm{A}}} d(\mathbf{u}^{i[k+r](n)\bigstar}, \mathbf{u}^{i*(n)}) \tag{10}$$

IV. Case Study

This section presents the proposed method's test results.

1) Test systems: We use the publicly available IEEE 57-bus system data set, available via the MATPOWER [12]. The IEEE-57 bus system is decomposed into 2 regions as in [11].

2) Training data: IEEE 57-bus system is used as the seed information to generate 72 data points (with a train/test ratio of 2:1). We consider a 5-percentage fluctuation of each load node. And we consider 100 ADMM iterations, i.e., $N_{\mathrm{K}} = 100$.

3) ADMM configuration: We initialize the ADMM values $\mathbf{u}^{i[k=0](n)}$, $\boldsymbol{\lambda}^{[k=0](n)}$ to zero. We use $\rho = 5000$ for ADMM. And we consider 3 recurrent steps, i.e., $N_{\mathrm{R}} = 3$.

4) Neural network configuration: We adopt one hidden layer for each neural network with 128 hidden units, and use the Rectified Linear Unit (ReLU) for the nonlinear activations. The output layer uses the Hyperbolic Tangent (TanH) activations to ensure $\hat{\mathbf{u}}^i_{\mathtt{Ind}} \in \mathcal{B}$, where $\mathcal{B}$ is the ℓ_∞ unit ball.

We apply the proposed learning method after running ADMM for 10 iterations to obtain inputs for the neural networks. Figures 6 and 7 showcase the testing results using ADMM and the proposed method from two perspectives; optimality and feasibility. Note that the feasibility of local constraints is enforced for both ADMM and the proposed method. Therefore, we only measure violations of coupled constraints. Our method significantly improves the convergence speed ($5\times$ fewer iterations) and the feasibility criteria.

TABLE I
Average running time of one iteration using ADMM solvers and the proposed learning method.

Method	Time(ms)
Proposed method	0.2973
ADMM using ECOS [13]	29.97
ADMM using CVXOPT [14]	103.9
ADMM using OSQP [15]	14.87

Table I compares the average run-time of one ADMM iteration against the proposed learning-based method. The proposed solution speeds up the solution time of each ADMM iteration by up to $50\times$. Furthermore, given that our method needs fewer iterations to converge, the overall convergence time will be significantly shorter as well ($\sim 250\times$ faster).

978-1-6654-6441-3/23/$31.00 ©2023 IEEE
Authorized licensed use limited to: FH Technikum Wien. Downloaded on May 11,2026 at 14:32:48 UTC from IEEE Xplore. Restrictions apply.

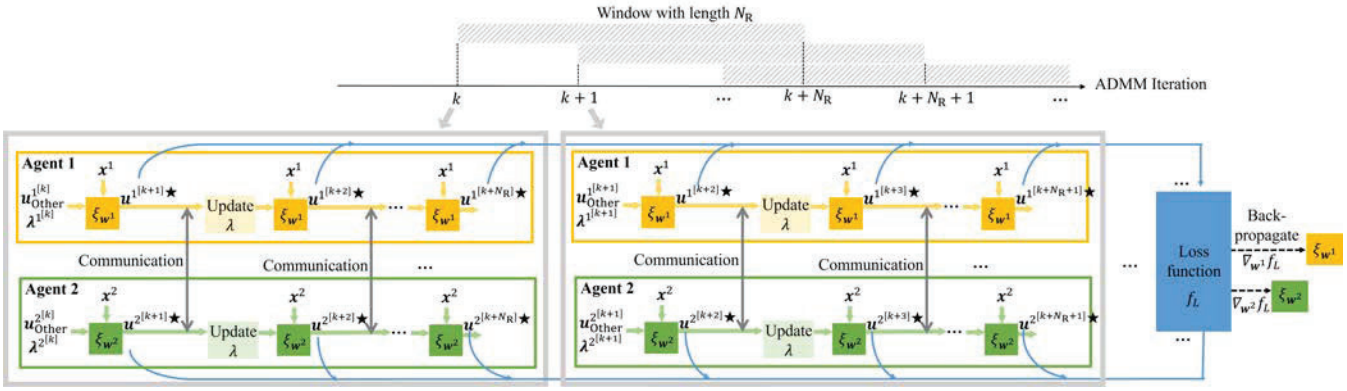

Fig. 5. Illustration of the training process for the two agent example presented in Fig.1. The prediction of converged ADMM values is a time-series prediction problem, and hence we adopt a recurrent training method where we consider the future output over a window with N_{R} length. Thus, at each ADMM iteration, we will minimize the loss over N_{R} horizon. Finally, the loss function over all iterations and all recurrent steps (N_{R} recurrent steps for each iteration) is computed to update the parameters $\mathbf{w}^i$ in $\xi_{\mathbf{w}^i}$ using back-propagation.

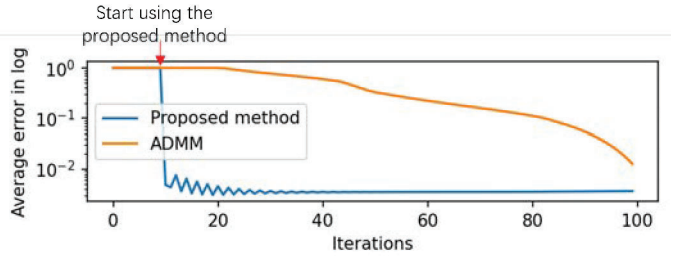

Fig. 6. Error comparison between ADMM and the proposed method. The y-axis shows the average error percentage in log, i.e., $\log_{10}\left(\frac{1}{N}\sum_{n=1}^{N}\frac{\sum_{i\in\mathcal{N}_{\mathrm{A}}}F^i(\mathbf{u}^{i[k](n)})-\sum_{i\in\mathcal{N}_{\mathrm{A}}}F^i(\mathbf{u}^{i[k](n)*})}{\sum_{i\in\mathcal{N}_{\mathrm{A}}}F^i(\mathbf{u}^{i[k](n)*})}\right)$.

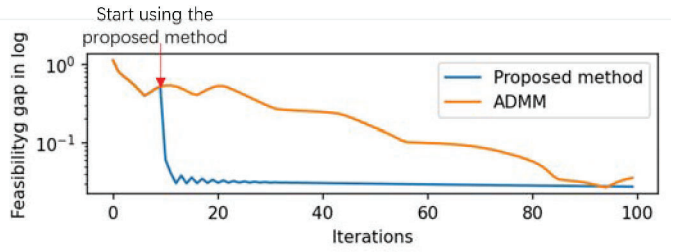

Fig. 7. Feasibility gap using ADMM and the proposed method. The y-axis shows the average feasibility gap in log, i.e., $\log_{10}\left(\frac{1}{N}\sum_{n=1}^{N}\sum_{\text{all } b,\bar{b}}|\theta^{b,\bar{b}[k](n)}_{\text{Copy}}-\theta^{\bar{b}[k](n)}|\right)$.

V. Conclusion

We propose a learning method to accelerate the convergence of ADMM for distributed DC-OPF where agents predict the convergence value of local variables by communicating with neighbors. In the proposed method, local neural networks are trained and optimized to maximize the overall convergence. The proposed method does not require post-processing steps to improve feasibility because local constraints are enforced using a gauge mapping method and coupled constraints are penalized during ADMM iterations. The proposed method not only reduces the number of ADMM iterations needed to converge but also speeds up the solution time of each ADMM iteration significantly.

References

[1] Erseghe, Tomaso. "Distributed optimal power flow using ADMM." IEEE transactions on power systems 29.5 (2014): 2370-2380.

[2] Li, Meiyi, Soheil Kolouri, and Javad Mohammadi. "Learning to Solve Optimization Problems with Hard Linear Constraints." arXiv preprint arXiv:2208.10611 (2022).

[3] Pan, Xiang, et al. "Deepopf: A deep neural network approach for security-constrained dc optimal power flow." IEEE Transactions on Power Systems 36.3 (2020): 1725-1735.

[4] Park, Seonho, and Pascal Van Hentenryck. "Self-Supervised Primal-Dual Learning for Constrained Optimization." arXiv preprint arXiv:2208.09046 (2022).

[5] Chen, Yize, Ling Zhang, and Baosen Zhang. "Learning to solve DCOPF: A duality approach." Electric Power Systems Research 213 (2022): 108595.

[6] Torre, Paul Serna, and Patricia Hidalgo-Gonzalez. "Decentralized Optimal Power Flow for time-varying network topologies using machine learning." Electric Power Systems Research 212 (2022): 108575.

[7] Dobbe, Roel, et al. "Toward distributed energy services: Decentralizing optimal power flow with machine learning." IEEE Transactions on Smart Grid 11.2 (2019): 1296-1306.

[8] Lotfi, Mohamed, et al. "A fully decentralized machine learning algorithm for optimal power flow with cooperative information exchange." International Journal of Electrical Power and Energy Systems 139 (2022): 107990.

[9] Biagioni, David, et al. "Learning-accelerated ADMM for distributed DC optimal power flow." IEEE Control Systems Letters 6 (2020): 1-6.

[10] Mak, Terrence WK, et al. "Learning Regionally Decentralized AC Optimal Power Flows with ADMM." arXiv preprint arXiv:2205.03787 (2022).

[11] Baker, Kyri, et al. "Distributed MPC for efficient coordination of storage and renewable energy sources across control areas." IEEE Transactions on Smart Grid 7.2 (2016): 992-1001.

[12] Zimmerman, Ray Daniel, Carlos Edmundo Murillo-Sánchez, and Robert John Thomas. "MATPOWER: Steady-state operations, planning, and analysis tools for power systems research and education." IEEE Transactions on power systems 26.1 (2010): 12-19.

[13] Domahidi, Alexander, Eric Chu, and Stephen Boyd. "ECOS: An SOCP solver for embedded systems." 2013 European Control Conference (ECC). IEEE, 2013.

[14] Andersen, Martin, Joachim Dahl, and Lieven Vandenberghe. "CVXOPT: Convex optimization." Astrophysics Source Code Library (2020): ascl-2008.

[15] Stellato, Bartolomeo, et al. "OSQP: An operator splitting solver for quadratic programs." Mathematical Programming Computation 12.4 (2020): 637-672.

978-1-6654-6441-3/23/$31.00 ©2023 IEEE
Authorized licensed use limited to: FH Technikum Wien. Downloaded on May 11,2026 at 14:32:48 UTC from IEEE Xplore. Restrictions apply.